REPORT · 2026年10月

缺陷分析与修复

M1 耗时与内存爆炸诊断

文保数字化研发组
2026年10月

目 录

一、实机现象 3

二、定位过程 4

三、根因：聚类半径与点密度脱钩 5

四、修复：三重保险 6

4.1 尺度基准改为 MPS 6

4.2 聚类只在降采样工作副本上做，标签回投全分辨率 6

4.3 邻域表预算看门狗 6

4.4 配套改动 6

五、验证 7

5.1 同一份真实数据的修复前后对比 7

5.2 全链路实跑（同一项目，S1 走断点续跑） 7

六、S1 那 30 分钟花在哪里 8

6.1 可用的三个旋钮（按性价比排序） 8

七、结论与后续 9

一、实机现象

2026-10-23 实机，一件真实中小型文物（30 张 4496×3000 照片，纯照片场景②，无激光扫描件）跑全链路：S1 耗时 30 分钟正常结束；M1 连续运行约 50 分钟仍未完成，进程常驻内存增长到约 10 GB，最终被人工取消。但用户查看中途产出的点云 PLY，几何形态本身是可用的。

日志停在这一行之后再无任何输出：

```
21:10:34 [INFO] [h3d.runner.batch_processor] M1 开始：relic_001
21:10:34 [INFO] [h3d.io.pointcloud_io] 读取点云 ...fused_dense.ply：677274 点
（此后 50 分钟无输出）
```

关键条件：M1 的输入是 SfM 稠密重建产物 **fused_dense.ply，677274 点 / 34.5 MB**，包围盒对角线 diag = 15.3546（SfM 任意单位）。

二、定位过程

M1 在纯照片场景下会依次做：读取稠密点云 → **背景/主簇剔除** → 体素降采样 → 统计去噪 → 法线估计与一致化 → 落盘。把每个子步骤单独拎出来在**同一份真实数据**上计时，结果如下。

子步骤	输入点数	实测耗时	峰值内存
读取 PLY	677274	0.2 s	0.14 GB
深拷贝 PointCloud	677274	0.0 s	0.19 GB
cluster_dbscan(eps=0.02×diag)	677274	> 284 s 未结束（手工终止）	4.6 GB 且持续增长
（对照）体素降采样 + 去噪 + 法线	70955	2.5 s	0.18 GB
（对照）cluster_dbscan 在降采样后	70955	1.4 s	0.18 GB

结论：**瓶颈唯一且明确**——背景主簇剔除里的 DBSCAN，而且它跑在**未降采样的 677k 原始稠密点云**上。内存以约 45 MB/s 线性增长、日志无任何输出，与用户观察到的「50 分钟 / 10 GB / 看着像卡死」完全吻合。

三、根因：聚类半径与点密度脱钩

背景剔除的原实现用包围盒对角线作尺度基准：

```
eps = cluster_eps_frac * diag = 0.02 * 15.3546 = 0.3071
```

而这份点云的平均点间距（MPS，最近邻距离中位数）只有 **0.00299**，于是 eps = **102 倍 MPS**：半径里塞进的是一整片曲面。实测每点邻域：

eps 取值	相对 MPS	每点平均邻域点数
0.3071（现行）	102 ×	279（中位 319，最大 416）
0.0299（10 × MPS）	10 ×	105
0.0180（6 × MPS）	6 ×	31
0.0120（4 × MPS）	4 ×	17

Open3D 的 cluster_dbscan 是单线程实现，且要为每个点建立完整的邻域表：677274 × 279 ≈ **1.9 亿条边**，每条边一个 int 再加上 67 万个各自独立堆分配的 vector 头，内存以 GB 计、时间以十分钟计。

> **为什么以前没暴露**：同一条代码在 6 万点的旧数据上只要 1.4 秒。成本随「点数 × 平均邻域」增长，而邻域本身又随密度上升——照片越多、重建越细，这件事只会越来越糟。这是**算法复杂度层面的缺陷**，不是参数没调好。

四、修复：三重保险

4.1 尺度基准改为 MPS

聚类半径改为「对角线上限 + 点间距基准」双约束：

```
eps = min(cluster_eps_frac * diag, cluster_eps_k * MPS)   # cluster_eps_k 默认 10
```

密度越高的点云 eps 自动越小，邻域规模被钉在 ~πk² 量级（k=10 时约 300），与包围盒大小解耦。

4.2 聚类只在降采样工作副本上做，标签回投全分辨率

聚类只用来判断「哪些点属于同一件器物」，不需要全分辨率。点数超过 bg_max_cluster_points（默认 15 万）时，先体素降采样到上限以内再聚类，然后按最近邻把簇标签**回投**到全分辨率点云——

原始点一个不丢，输出仍是全分辨率的背景剔除结果，而 DBSCAN 看到的点数被钉死在常数级。

4.3 邻域表预算看门狗

聚类前抽样 300 点估算邻域表规模 n × avg_neighbors。超过 6×10^7 条边就逐级收紧半径（k×MPS → 4×MPS → 2×MPS）；若仍然超预算才放弃聚类，并且**必须**写 metrics + WARNING，绝不做「悄悄跳过」——跳过等于背景剔除去失效，用户又会看到「白花花一片、看不出是器物」。

4.4 配套改动

改动	文件	原因
M1 打印背景剔除摘要（eps、MPS 倍数、聚类点数、耗时）	runner/__init__.py	原来这一步完全静默，出问题无法定位
新增 bg_cluster_eps_k / bg_max_cluster_points 配置项	core/config.py + configs/example.yaml	不同文物密度差异大，需可调
S1 增加分段计时（稀疏 / 稠密）与三段 COLMAP 计时	sfm_solve.py	30 分钟只有一个总数，无法判断该优化哪一段
稠密段计入 S1 耗时	sfm_solve.py	原实现 elapsed 在稠密之前取，22 分钟的稠密重建从耗时里消失了
matcher 支持 auto，配置显式值不再被标定覆盖	core/config.py + sfm_solve.py + gui/config_edit.py	用户配置写 sequential，标定却悄悄换成 exhaustive（配对数 58→435）

五、验证

5.1 同一份真实数据的修复前后对比

指标	修复前	修复后
677k 点背景主簇剔除	284 s 未结束（已 4.6 GB）	4.9 s
峰值内存	4.6 GB 且持续增长（实机 10 GB）	0.23 GB
M1 全阶段	50 分钟未完成	6.28 s
保留点数	—	622157 / 677274（91.9%）
保留包围盒对角线	15.35（被子板碎片撑大）	5.95

聚类结果：84 个簇，保留 1 个主簇（91.9% 的点），背景衬板与桌面碎片被剔除，包围盒从 15.35 收回到 5.95——与人工目视判断的器物尺度一致。

5.2 全链路实跑（同一项目，S1 走断点续跑）

阶段	耗时	产出
S1（跳过）	—	复用 2 个产物
M1	6.28 s	41485 点（10_clean）
M2	0.03 s	单站，恒等变换落盘
M3	2.09 s	59107 顶点 / 117679 面
M4	98.67 s	OBJ + MTL + GLB + 2048² 贴图
M5	0.78 s	coverage 82.4%，ok=True
M6	0.42 s	3 格式交付 + 清单
合计	约 109 s	done=1 failed=0

交付产物落在 80_export/relic_001/：ply / obj / mtl / glb / diffuse.png(4.6 MB) / accuracy.csv / 论文表格。

六、S1 那 30 分钟花在哪里

COLMAP 自己在 99_runs/sfm_tmp/<编号>/logs/ 下写了每步的 timer 日志，可直接归因（30 张照片，逐条实测）：

步骤	COLMAP 命令	耗时	占比
1 特征提取	feature_extractor	1.62 min	5%
2 特征匹配	exhaustive_matcher（435 对）	4.57 min	15%
3 稀疏建图	mapper	0.72 min	2%
4 去畸变	image_undistorter	0.04 min	0.1%
5 稠密立体匹配	patch_match_stereo（两遍）	22.63 min	76%
6 深度图融合	stereo_fusion	0.13 min	0.4%
合计	—	29.7 min	100%

稠密立体匹配一段独占 76%。它跑两遍（光度一致性 + 几何一致性），在 2400 px 长边、30 个视角上做逐像素的块匹配，是 GPU 密集但仍是分钟级的量。

6.1 可用的三个旋钮（按性价比排序）

配置项	现值	建议	预计效果	代价
dense_geom_consistency	true	false	稠密段减半 → 约 11 min	fusion 改用光度深度图，弱纹理区噪点略增
dense_max_image_size	1600（默认）	1200	像素面积 ×0.56 → 约 12.7 min	点云细节下降，677k→约 380k 点
matcher	auto（30 张→exhaustive）	sequential	匹配 4.57 → 约 0.7 min	环形拍摄首尾闭环依赖 loop detection，注册数可能下降

两者同时开（geom_consistency=false + 1200 px）稠密段可压到 3~4 分钟，S1 总时长从 30 分钟降到 **7 分钟**量级；但分辨率一旦降下来就补不回去，建议**先用全分辨率出一版成果，再按需要降档做参数试验。**

七、结论与后续

1. 本次「50 分钟 + 10 GB」是**单一缺陷**造成的：背景主簇剔除的聚类半径只跟包围盒对角线挂钩，与点云密度脱钩，在 677k 点的稠密重建结果上退化成上亿条边的邻域表。修复后 M1 从「跑不完」变为 6.28 秒，内存从 10 GB 降到 0.23 GB，且剔除效果更好（包围盒 15.35 → 5.95）。

2. S1 的 30 分钟**不是缺陷**：76% 是稠密立体匹配的正常开销，已补充分段计时，后续可在报表里直接看到稀疏/稠密各占多少。

3. 待办：M4 的 98.7 秒（UV 展开 33 s + 投影采样）是下一个大头；镂空与手臂内侧仍缺照片，Poisson 会产生空洞，需补拍。

> 一条通用经验：**凡是以包围盒对角线（或任何几何极值）作为尺度基准的参数，都必须同时受点间距约束。**几何极值只描述「多大」，点间距才描述「多密」；只按大小定半径，在稠密数据上必然过冲。